

Composite

jueves, 9 de abril de 2026 12:39

- **Composite**
es un patrón de **diseño estructural**
Quiere representar jerarquías parte-todo de objetos(fundamental encontrar un polimorfismo entre ellos). **Permite tratar objetos individuales y grupos de objetos de la misma manera**

Component:
¿Qué es?: es la interfaz o clases abstracta base
-Define operaciones comunes
-Puede tener comportamiento por defecto
-Declara métodos para manejar hijos(opcional)
(Todos los leaf (objeto simple)y composite(objeto compuesto) comparten esta interfaz)

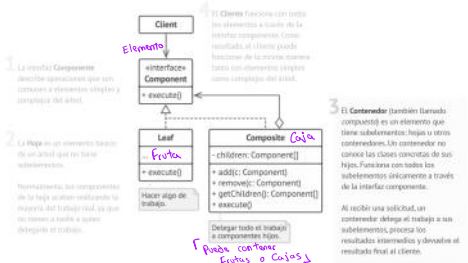
Leaf:
¿Qué es?: Representa objetos simples
-No tiene hijos
-implementa directamente el comportamiento

Composite:
¿Qué es?: representa objetos complejos
-Puede tener hijos(de tipo leaf o composite)
-Implementa operaciones sobre esos hijos **si no está el patrón está mal implementado**
-Maneja colecciones(lista,array,etc)

Consecuencia
-> Composición recursiva: un objeto puede contener otros y esos a su vez otro,su funcionamiento es recursivo, ya que los objetos compuestos **delegan operaciones a sus hijos**.
->Transparencia para cliente: usa la misma interfaz para leaf o composite
->Facil Extension: puedes agregar nuevo tipos sin cambiar el codigo cliente

- Malos Olores**
- jerarquía no es polimórfica
 - Composite no tiene relación con otras clases de la jerarquía
 - Composite no implementa add/remove
 - Composite no delega en la colección

¿Cuándo usarlo?
Cuando hay estructuras jerárquicas (árbol)
Cuando quieras tratar objetos individuales y compuestos igual
Cuando hay composición recursiva
Ejemplos:
Archivos/carpetas
Productos/combo
UI (grupos de elementos)



```
// Componente
interface Elemento {
    void asbtract mostrar();
}

// Hoja
class Fruta implements Elemento {
    private String nombre;
    public Fruta(String nombre) { this.nombre = nombre; }
    public void mostrar() { System.out.println("Fruta: " + nombre); }
}

// Compuesto
class Caja implements Elemento {
    private List<Elemento> elementos = new ArrayList<>();

    public void agregar(Elemento e) { elementos.add(e); }

    public void mostrar() {
        System.out.println("Caja con:");
        for (Elemento e : elementos) {
            e.mostrar();
        }
    }
}
```